Project 10: Job Portal & Applicant Tracking System

Intermediate • 60-72 hours (3-4 weeks part-time) • Deployment complexity: Intermediate

Difficulty Level	Intermediate | Deployment complexity: Intermediate
Description	A two-sided platform where candidates build profiles, upload resumes and apply to jobs, while recruiters post openings and move applicants through a hiring pipeline. Role separation runs through every layer, from route guards to database queries. Resume upload, search with multiple facets and status-change notifications make it a realistic product. It mirrors the placement-training domain, which makes for a persuasive interview narrative.
Learning Objectives	• Multi-role architecture: candidate, recruiter and admin with distinct data access • Complex faceted search and filter APIs with compound indexes • Secure file upload, storage and controlled access to private documents • Application state machine with permitted transitions enforced server-side • Aggregation pipelines for recruiter analytics and funnel reporting • Email notifications on status change with templated content • Preventing IDOR - users must not access resources by guessing IDs
Key Features	• Candidate: profile builder, resume upload, skills, education, experience, portfolio links • Job search with filters for role, location, experience, CTC band, work mode and skills • One-click apply with resume selection, cover note and duplicate-application prevention • Application tracker with pipeline stages: applied, shortlisted, interview, offered, rejected • Recruiter: post and edit jobs, view ranked applicants, change stage, add internal notes • Saved jobs, job alerts and profile-completion scoring • Admin: approve job posts, manage users, platform analytics
Recommended Tech Stack	Frontend: React 18 (Vite), Redux Toolkit or Zustand, React Router v6, Tailwind CSS or Ant Design, react-hook-form + Zod, react-dropzone. Backend: Node.js, Express, Mongoose, multer + Cloudinary/S3, Nodemailer, JWT with role middleware, express-validator. Database: MongoDB Atlas
Core Data Models	• `User { name, email, passwordHash, role (candidate/recruiter/admin), companyId }` • `CandidateProfile { userId, headline, skills[] (indexed), experienceYears, education[], resumes[{url, name, uploadedAt}], expectedCtc }` • `Job { recruiterId, companyId, title, description, skills[], location, workMode, minExp, ctcRange, status, deadline, createdAt }` with compound indexes • `Application { jobId, candidateId, resumeUrl, coverNote, stage, stageHistory[], recruiterNotes[] }` with a unique index on `{ jobId, candidateId }`
Key API Endpoints	• `GET /api/jobs?role=&location=&minExp=&ctc=&skills=&page=` (faceted search) • `POST /api/jobs` (recruiter only) | `PATCH /api/jobs/:id` | `PATCH /api/jobs/:id/close` • `POST /api/applications` (unique index prevents duplicates) | `GET /api/applications/me`

	• `GET /api/jobs/:id/applicants (recruiter of that job only) | PATCH /api/applications/:id/stage` • `POST /api/uploads/resume | GET /api/files/:id (signed, access-controlled download)`
Third-Party APIs / Services	Cloudinary or AWS S3 for resume storage, SendGrid or Nodemailer for notifications, optional Arbeitnow/Remotive APIs to seed job data, Google Places API for location autocomplete
Estimated Build Time	60-72 hours (3-4 weeks part-time)
Deployment Blueprint	• Vercel frontend, Render backend, MongoDB Atlas database, S3 or Cloudinary for resumes • Resumes are private documents - store them with restricted access and serve via short-lived signed URLs, never public links • Set upload constraints: PDF/DOCX only, maximum 2 MB, validated by mime type on the server, not just the client • Environment separation: staging with seeded demo data, production with real accounts • Enable helmet, CORS whitelisting, rate limiting on login and apply endpoints, and express-mongo-sanitize • Document demo credentials for both roles in the README so reviewers can test both dashboards • Post-deploy: verify that a candidate cannot open another candidate's resume URL or applicant list
Industry Relevance	Two-sided marketplaces with role-based pipelines are a standard enterprise architecture used in HR tech, insurance and banking workflows. The access-control angle lets a candidate discuss security concretely.
Placement Interview Angle	• How do you enforce that a recruiter can only view applicants for jobs they own? • What is IDOR, and how did you prevent it on the resume download endpoint? • How do you prevent duplicate applications? Why is a unique index better than a find-then-insert check? • Design the indexes for your job-search query. Which order and why? • Why are resumes served through signed URLs rather than public Cloudinary links? • How do you model and enforce valid application-stage transitions?
Stretch Goals	• Add a resume parser and automatic keyword match score against the job description • Add real-time application-status notifications with Socket.io • Add interview scheduling with calendar invites and reminder emails • Add Redis caching for the job-search endpoint with tag-based invalidation
Evaluation Rubric Hint	Security: every resource endpoint must verify ownership, and private files must not be publicly reachable by URL. Deployment: both role dashboards must be demonstrable on the live URL with seeded credentials.